

Test 1 Schedule — Multi-Sport Prop Model Build Plan

Start Date: August 19, 2026

Target: NFL Model Live by September 1, 2027

Phase 1: Tennis Foundation (Weeks 1-8, Aug 19 – Oct 14, 2026)

Build the core pipeline on tennis since it has the fewest moving parts — no roster, no injury-ripple effects, no possession sharing.

- **Weeks 1-2:** Data acquisition and schema design. Pull Jeff Sackmann's ATP/WTB historical match archives (free, decades of data), design player/match/prop tables as a simplified version of the NBA schema (players, matches, surfaces, odds, playerproppredictions).
- **Weeks 3-4:** Feature engineering. Build rolling averages (last 3/5/10 matches), surface-specific splits, head-to-head history, rest days between matches — the tennis analogs to the NBA rolling-average and rest-day features.
- **Weeks 5-6:** Baseline model + GBM model. Build the interpretable rolling-average baseline first, then the LightGBM/XGBoost model with quantile regression for confidence intervals.
- **Weeks 7-8:** Ensemble + backtesting harness. Build the weighted-average or stacked ensemble, and — critically — build the time-based train/test split and backtesting evaluation loop (MAE, RMSE, edge-vs-market) as a reusable module, not a one-off script.

Why this matters for translatability: the backtesting harness, model-versioning convention (modelversion, predictiontimestamp), and ensemble-weighting logic built here should be sport-agnostic from day one — treat tennis as the "reference implementation" of a generic interface, not a tennis-specific system.

Phase 2: Intensive Testing and Iteration on Tennis (Weeks 9-14, Oct 15 – Nov 25, 2026)

This is where you stress-test rather than build.

- **Weeks 9-10:** Run full historical backtests across multiple tennis seasons, comparing baseline vs. GBM vs. ensemble vs. market line. Tune ensemble weights based on out-of-sample MAE.
- **Weeks 11-12:** Add the optional Bayesian/hierarchical layer (player random effects) to see how much it improves low-data cases (e.g., players with short match histories) — this validates the "postpone if too heavy" decision point from the original NBA plan.
- **Weeks 13-14:** Stress-test edge cases: injuries/retirements mid-match, surface transitions (clay-to-grass season changes), new players with sparse history. Document every failure mode — this becomes the checklist for the next sport.

Milestone (Nov 25, 2026): A validated, backtested tennis model with documented accuracy metrics and a clean, modular codebase.

Phase 3: Port to a Team Sport — Refactor for Team Structure (Weeks 15-24, Nov 26, 2026 – Jan 27, 2027)

Pick one team sport to force solving the "team" abstraction before jumping to NFL specifically. Baseball (MLB) is the better bridge sport here rather than NBA, because pitcher-vs-batter is still largely a 1v1 matchup layered on top of team context — it's the smallest incremental step up in complexity from tennis, and Statcast/Lahman data is free.

- **Weeks 15-17:** Extend the schema to add teams, schedules, and rosters. This is the key translatability investment — design the games/teams tables generically (home/away, schedule, rest days, travel) so the same structure will later serve NFL without a rebuild.
- **Weeks 18-20:** Add team-context features (opponent strength, park/venue factors, pace-equivalent metrics) on top of the existing player-level feature pipeline. Reuse the baseline/GBM/ensemble code almost unchanged — only the feature inputs change.
- **Weeks 21-22:** Add the injury/teammate-availability feature layer (e.g., "is the starting pitcher a late scratch") — this is the piece tennis didn't need and NFL will need heavily (a starting QB being out is a massive signal).
- **Weeks 23-24:** Full backtest and validation cycle on the team sport, same rigor as tennis.

Milestone (Jan 27, 2027): A working team-sport model and — importantly — a documented "what changed" diff between tennis and the team sport, which becomes the translation playbook.

Phase 4: Generalize the Architecture (Weeks 25-30, Jan 28 – Mar 10, 2027)

Before jumping to NFL, spend real time making the system sport-agnostic rather than sport-specific.

- Refactor feature engineering into a config-driven pipeline (define per-sport feature sets in config files, not hardcoded logic).
- Abstract the model layer so baseline/GBM/ensemble/Bayesian components accept any sport's feature table without code changes — only hyperparameters and feature lists vary.
- Build a generic vwfact* reporting view template that can be parameterized per sport.
- Write internal documentation: "to add a new sport, do X, Y, Z" — this is the deliverable that makes NFL (and beyond) fast.

Milestone (Mar 10, 2027): A sport-agnostic framework, proven on two very different sports (individual + team), with a documented onboarding process for a third.

Phase 5: NFL-Specific Build (Weeks 31-42, Mar 11 – Jun 2, 2027)

Now the actual NFL work should be comparatively fast because the architecture is proven.

- **Weeks 31-33:** NFL data acquisition (free sources: nflverse/nflfastR data, ESPN/API-Sports free tiers) and schema mapping into the generalized framework.
 - **Weeks 34-36:** NFL-specific feature engineering — this is where real new work happens, since NFL has unique dynamics (small 17-game season limits historical depth per team, weather effects, injury reports are unusually influential, positional matchups like WR-vs-CB).
 - **Weeks 37-39:** Train and backtest baseline/GBM/ensemble on NFL data. Expect more tuning time here than tennis/MLB because NFL's small sample size per season makes overfitting a bigger risk — a well-documented challenge in the sports-forecasting literature (short seasons reduce statistical power).
 - **Weeks 40-42:** Full historical backtest across multiple NFL seasons, market-line comparison, edge/ROI simulation.
-

Phase 6: Buffer, Polish, and Dashboard Integration (Weeks 43-54, Jun 3 – Sept 1, 2027)

- **Weeks 43-46:** Dashboard integration — extend the existing reporting views and Dash/dashboard UI to support NFL alongside tennis/MLB with a sport-switcher.
 - **Weeks 47-50:** Live-pipeline testing — run the NFL model through a real preseason/early-season slate to validate the daily ETL, feature refresh, and scoring pipeline under real operational conditions (not just historical backtests).
 - **Weeks 51-54:** Buffer for the inevitable: data source hiccups, model recalibration, bug fixes discovered only under live conditions. This buffer is deliberate — every phase before this has zero slack built in, and live sports data pipelines routinely surface issues backtests don't catch (API rate limits, schema drift, odds-feed formatting changes).
-

Why This Sequencing Protects the Translatability Goal

Going tennis → MLB → generalize → NFL (rather than tennis → NFL directly) is what actually makes the NFL build fast. Jumping straight from tennis to NFL would mean solving "add team structure" and "add NFL's idiosyncrasies" simultaneously, with no intermediate checkpoint to catch design mistakes. The MLB step is deliberately chosen as the smallest possible bridge — it forces building team/schedule/injury abstractions once, in a lower-stakes sport, before applying them to NFL where a bad abstraction is more costly to fix. The generalization phase (Phase 4) is the one most teams skip, and it's the single highest-leverage phase for the stated goal of "seamlessly translate it to other sports" — budget real time for it rather than treating it as free.